Week 2: Building the Backend: MongoDB and Mongoose

Problem Statement

This week we begin building the backend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

Your job is to write **Mongoose queries** that power the data layer of this platform and to use **GitHub Copilot** to assist you in AI-driven workflows for database design, seeding, and testing.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
3. As you make changes throughout this tutorial, remember to regularly commit your work.

The project structure is as follows:

```
├── models/           → Mongoose models for User, Subreddit, Thread
├── data/             → Seed JSON files (users, subreddits, threads) for
populating the database
├── populate_db.js    → Script to load seed data into MongoDB
└── queries.js        → File where you will implement the Mongoose queries
for this session
```

This week's tasks are divided into two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Mongoose Queries	No AI Tools
Part 2	AI-Assisted Development	Implementation, Testing and DB Design	GitHub Copilot

By the end of this session, you will be able to:

- Write Mongoose queries for common DB operations and patterns
- Understand System Prompts, User Prompts, and the Context Window
- Use Copilot's **Agent Mode** to implement DB Schemas, generate test data and test scripts
- Use Copilot's **Plan Mode** for brainstorming and design tasks

Part 1: Manual Implementation — Setup and Queries

Step 1.1: Project Setup

1. Create a `.env` file in the root of the assignment folder. Add the following entry in the file:

```
MONGODB_URI=your_mongodb_atlas_connection_string
```

You need a MongoDB Atlas account and a free cluster. If you haven't set one up yet, go to mongodb.com/atlas and create a free cluster. Copy the connection string and paste it as the value above.

2. Install dependencies:

```
npm install
```

3. Seed the database with the provided sample data:

```
npm run populate
```

This runs `populate_db.js`, which clears any existing data and loads fresh records from `data/users.json`, `data/subreddits.json`, and `data/threads.json`. Verify that your MongoDB cluster now contains the seeded data.

4. Run your queries at any point with:

```
npm run queries
```

Open `queries.js` and uncomment the queries you want to run inside `runQueries()`.

Step 1.2: Understand the Schemas and Relationships

Before writing queries, take a moment to review the three DB schemas in `models/` and understand their relationships.

Step 1.3: Implement the Queries

Implement the following queries in the file `queries.js`:

1. Find user by email diana@example.com
2. Get threads in a subreddit programming

3. Threads posted by a specific user Ethan
4. Users who posted threads
5. Threads with at least 2 upvotes
6. Threads created on or after January 1, 2024
7. Add a new thread (Create) Subreddit devops author Ethan
8. Update title of the thread "Docker and kubernetes?"
9. Delete all threads by Ethan
10. Delete all subreddits and their associated threads
11. Retrieve the ids of all users who have posted threads, along with the number of threads they have posted
12. Find the author ID and thread count for the user who posted the most threads

Each query should be implemented inside the corresponding function. After implementing a query, uncomment the corresponding function call in `runQueries()` and run `npm run queries` to verify the output.

Part 2: AI-Assisted Development — GitHub Copilot

Before we dive into specific AI use cases for building the backend, let us first understand some foundational concepts that will help us interact effectively with AI tools like GitHub Copilot.

2.1 System Prompts vs User Prompts

When interacting with AI models, understanding the distinction between these two prompt types helps you get better, more consistent results.

System Prompt

A system prompt is a high-priority instruction given to the AI that controls how it behaves throughout the entire conversation. It's a hidden set of directives that sets the tone, defines constraints, and enforces standards — like coding style or security rules. You typically can't see or edit it directly in tools like GitHub Copilot, but it shapes every response the AI gives.

For example, a simple system prompt for a JavaScript coding assistant might look like:

```
You are a helpful AI coding assistant.
Follow secure coding practices.
Do not expose secrets.
Prefer modern ES6+ syntax.
```

User Prompt

A **User Prompt** is what **you** type in the chat interface. It's the input you send to the AI during a conversation, and it typically includes:

- Task instructions
- Constraints
- Context — file references, selected code
- Examples
- Desired output format

The AI generates its response based on the combination of the system prompt, all previous messages in the conversation, and your latest user prompt.

2.2 What Is Inside the Context Window?

The **Context Window** is the total amount of information the model can "see" at once. It typically includes:

- The system message
- All previous user prompts and AI responses in the current conversation
- Your latest user prompt
- Any files or code you've attached
- Any tools used by the agent (file explorer, code interpreter, etc.)

As the conversation grows, the context window fills up. If it becomes too large, the model may start to "forget" earlier parts of the conversation, leading to less relevant or inconsistent responses. It's generally recommended to keep context window usage below 50% for best results.

As the conversation grows, more information is added to the context window. While modern models support large context windows, as context increases, models may struggle to effectively use all the information — a phenomenon often called context degradation. This can lead to issues such as ignoring earlier instructions, missing important details, or producing less consistent outputs.

Exercise: In GitHub Copilot Chat, open the model selector dropdown and go to **Manage Models**. Note the context window sizes for the available models. During a chat session, you can also track usage via the indicator at the top right of the chat interface.

Practical Context Management Tips

- Start a **new chat** for each new task — don't carry over unrelated context
- Only attach files that are directly relevant to the current task
- Use **Plan Mode before Agent Mode** for large or multi-step tasks — planning in a focused chat keeps context clean before execution begins
- Keep an eye on the context window usage. If it gets too high, use the /compact command to summarize and reduce context, or start a new chat.

2.3 Plan Mode

Plan Mode is a GitHub Copilot feature that lets you think through complex tasks before executing them. It helps you break work into smaller steps and validate the approach before any code is written or files are changed.

Use Plan Mode when:

- Brainstorming features or design decisions
- Designing database schemas
- Planning API architecture
- Preparing a refactoring or testing strategy

Always use a strong reasoning model in Plan Mode (Claude Opus 4.5+, GPT-5.2+, or Gemini 2+) to get the best quality plans. Carefully review the output and adjust before switching to Agent Mode for implementation.

2.4 AI Use Cases for Building the Backend DB

Let us now explore specific use cases where AI can assist you in building the backend for ThreadHive. These examples will show you how to leverage both Plan Mode and Agent Mode effectively.

A. Recreate Mongoose Models (Agent Mode)

In Part 1, you worked with pre-built Mongoose models and seed data. Now let's see if Copilot can recreate them from a textual description.

1. Commit your current code so your work from Part 1 is saved.
2. Delete all files inside the `models/` and `data/` folders. Keep the folders themselves — just remove the files inside them. We will recreate the models and seed data using AI.
3. Start a **new chat in Agent Mode** and provide the following prompt:

```
Create Mongoose models based on the following schema definitions. Each
model should be in its own file inside the /models folder. Use the
specified data types, constraints, and references.

**User**

| Field     | Type   | Constraints      |
| --------- | ------ | ---------------- |
| name      | String | required         |
| email     | String | required, unique |
| password  | String | required         |
| createdAt | Date   | required         |
```

```
**Subreddit**

| Field       | Type     | Constraints          |
| ----------- | -------- | -------------------- |
| name        | String   | required, unique     |
| description | String   | –                    |
| author      | ObjectId | required, ref → User |
| createdAt   | Date     | required             |

**Thread**

| Field     | Type     | Constraints                |
| --------- | -------- | -------------------------- |
| title     | String   | required                   |
| content   | String   | required                   |
| author    | ObjectId | required, ref → User       |
| subreddit | ObjectId | required, ref → Subreddit  |
| upvotes   | Number   | default 0                  |
| downvotes | Number   | default 0                  |
| voteCount | Number   | default 0                  |
| createdAt | Date     | required                   |
```

Review the generated models against these tables. Verify that field types, constraints, and references are correct before moving on.

4. Commit the generated models.

B. Generate a Seed Script (Agent Mode)

Now let us use AI to generate sample data for our newly created models. Start a **new chat in Agent Mode** and use this prompt:

```
Create a production-ready database seed script for the application.

CONTEXT:
- All Mongoose models already exist inside /models folder.
- The MongoDB connection string is stored in a .env file as the
MONGODB_URI variable.

Your task is as follows:
1. Create the following test data including:
   - 10 users
   - 6 subreddits
   - 20 threads, distributed across subreddits
   - Randomly assign authors to threads
   - CreatedAt timestamps spread across last 60 days
   - Random upvotes/downvotes
```

```
2. Maintain referential integrity — use ObjectIds correctly
3. Ensure all ObjectIds are unique
4. Create the sample data in separate json files for each model and save
them in the /data folder
5. Create a single script file to seed the database at /scripts/seed.js
6. The script must clear existing data at the start, insert data in proper
order, and log meaningful progress
```

Review the test data and ensure it meets the requirements. Review the generated seed script for correctness and completeness. After verifying, run the generated seed script and verify that the data is correctly inserted into the database.

C. Generate a Test Script

Let us now use AI to quickly test that our database models and queries are working as expected. This is a common workflow for developers — using AI to generate test scripts that validate their code.

Start a **new chat in Agent Mode** and use this prompt:

```
I want to create a script to test the CRUD operations of my MongoDB
database. I have already implemented the Mongoose models and seeded the
DB.

Help me create a set of 20 tests to test the CRUD operations of the
database. Do not use a testing framework — the tests should be implemented
in a standalone script in a single file located at /scripts/ that can be
run with Node.js.
```

Review the generated script, run it, and verify that the DB operations are functioning correctly.

> **Note:** The remaining two use cases (D and E) are **demonstration exercises only**. Their purpose is to show how AI can assist with brainstorming and architectural design. You will not be building the features or schemas generated in these sections — treat them as learning exercises for how to use Plan Mode effectively.

D. Brainstorm Features (Plan Mode)

Beyond generating code, AI tools are also valuable for brainstorming and high-level design. Let's explore how Plan Mode can help you think through features and architecture before writing any implementation code.

Open a **new chat in Plan Mode** and try this prompt:

```
I want to build a Reddit-like platform where users can share and discuss
content. Help me brainstorm a comprehensive list of features for this
application.

Organize the features by priority using the MoSCoW framework.

For each feature, include:
- User capability - What users can do with this feature
- Implementation complexity - Relative difficulty (Simple / Moderate /
Complex)
- Priority justification - Why it belongs in this tier

Think of interesting and creative features that would differentiate the
application and drive usage.
```

Let us select features that are most relevant and important from the generated list and save them as `features.md` in the root of the project. Switch to Agent Mode and give this follow-up prompt:

```
Save the Must have and the Should have features from the above list in a
markdown file named features.md in the root of the project.
```

Review the output and make any adjustments if necessary.

E. Design a MongoDB Schema (Plan Mode)

With your feature list in hand, use Copilot to design the database schema. Start a **new chat in Plan Mode**, attach `features.md`, and use this prompt:

```
As a senior backend architect and MongoDB expert help me design an optimal
MongoDB schema. I am building a Reddit-like application in the MERN stack.
I have attached a file #file:features.md that contains a detailed list of
application features.

Your task is to:
```

```
1. Carefully analyze the features.
2. Identify:
   - The core entities
   - Relationships between entities
   - Expected query patterns (read-heavy vs write-heavy)
3. Design an optimal MongoDB schema. For each collection include:
   - Fields
   - Data types
   - Relationships
   - Index recommendations (if any)

Outline two possible schema designs. Give the pros and cons of each and
recommend the best one based on the application features.
```

Study the generated designs and the reasoning behind them. This is a great example of how AI can assist not just with code generation but with high-level architectural decisions. You can even take the generated schema and implement it in Mongoose as an exercise.